

Лабораторная 2. Рекурсия, динамический массив, стек и дек

Идея работы

Здесь изучаются два способа организации вычислений. Рекурсия решает задачу через более маленькую версию той же задачи. Структуры данных задают, где хранится информация и какие действия с ней дешёвы. В работе реализуются факториал, два варианта Фибоначчи, Ханойские башни, массив с удвоением ёмкости, стек и дек.[1]

Рекурсия и стек вызовов

Факториал определяется как $n! = n \cdot (n-1)!$, а $0! = 1$. Вычисляя `factorial(4)`, программа приостанавливает умножение на 4, чтобы получить `factorial(3)`; затем приостанавливает умножение на 3 и так далее. Стек вызовов хранит локальные переменные и место возврата. Достигнув базового случая, вычисления возвращаются обратно: 1, 2, 6, 24.

У рекурсии должны быть базовый случай и продвижение к нему. Без первого вызовы не остановятся; без второго базовый случай может никогда не наступить. В Python обычно возникает `RecursionError` раньше исчерпания всей памяти. Отрицательные n в реализации отвергаются сразу. Для факториала выполняется $O(n)$ умножений и требуется $O(n)$ кадров стека; стоимость операций с большими числами рассматривается отдельно.

Наивное вычисление Фибоначчи использует $F(n) = F(n-1) + F(n-2)$, $F(0) = 0$, $F(1) = 1$. При $F(5)$ значение $F(3)$ вычисляется и внутри $F(4)$, и отдельно. Таких повторов быстро становится много. Число вызовов удовлетворяет $C(n) = C(n-1) + C(n-2) + 1$ и равно $2F(n+1) - 1$. Рост $O(\varphi^n)$, где $\varphi \approx 1,618$; $O(2^n)$ — более грубая верхняя граница.

Мемоизация хранит уже найденные значения в словаре. Каждый новый номер вычисляется один раз, а повторный вызов берёт ответ из кэша. В нашей функции счётчик включает и попадания в кэш: при $n \geq 2$ получается $2n - 1$ вызовов. Время по числу арифметических операций $O(n)$, память $O(n)$ для кэша и стека. Более компактный итеративный вариант мог бы хранить только два последних числа.

Ханойские башни

Есть три стержня и n дисков разного размера. Разрешено переносить верхний диск и нельзя класть больший на меньший. Чтобы перенести n дисков с A на C, сначала переносим $n-1$ с A на B, затем самый большой с A на C, затем $n-1$ с B на C.

Это даёт $H(n) = 2H(n-1) + 1$, $H(0) = 0$. Раскрывая рекурсию, получаем $H(n) = 2^n - 1$. Для трёх дисков нужно семь ходов. Генератор `hanoi` выдаёт ходы по одному: память самого генератора $O(n)$, а не $O(2^n)$, если не собирать все ходы в список. Тест не только считает ходы, но и воспроизводит стержни, проверяя законность каждого переноса.

Массив: размер и ёмкость — разные вещи

Размер `size` — сколько значений логически хранится. Ёмкость `capacity` — сколько ячеек выделено. Если `size < capacity`, `append` записывает значение в следующую свободную ячейку. Если места нет, создаётся буфер вдвое больше, старые значения копируются вручную, затем добавляется новое.

Для ёмкостей 1, 2, 4, 8 добавления выглядят так: первый элемент занимает уже выделенную ячейку; второй требует скопировать 1 элемент; третий — 2; пятый — 4. В реализации Python list используется только как фиксированный буфер [None] * capacity: его встроенный append не управляет ростом нашей структуры.

Инварианты: $0 \leq \text{size} \leq \text{capacity}$; длина буфера равна capacity; используемые индексы — от 0 до size-1. Метод pop очищает освобождённую ячейку, чтобы структура не удерживала ненужную ссылку. Отрицательные индексы здесь запрещены, хотя у встроенного list они разрешены: это явно выбранный контракт.

Почему append амортизировано $O(1)$

Для N добавлений число копирований не превышает $1+2+4+\dots < 2N$. Вместе с N записями новых элементов получается меньше $3N$ действий в модели «запись/копирование ссылки». Поэтому вся последовательность стоит $O(N)$, а одна операция амортизировано $O(1)$.

Банковское объяснение: за каждое добавление берём 3 условные единицы. Одну тратим на новую запись, две откладываем. За половину заполнения после прошлого роста накапливается достаточно кредита для следующего копирования. В CSV growth показаны $3N$ и фактическое $N+\text{copies}$: кредит не уходит в минус. Начальное выделение массива добавляет $O(1)$. Инициализация новых пустых буферов также суммарно линейна, но для её включения потребуются большой постоянный тариф.

Амортизированное время не равно среднему по случайным входам: оценка держится для любой последовательности добавлений из пустой структуры. Отдельная операция при росте по-прежнему $O(n)$.

Стек и дек

Стек — LIFO: последним положили, первым достали. Он построен на нашем DynamicArray. push добавляет в конец, pop удаляет с конца, peek читает верхний элемент. Примеры применения — отмена действий, проверка скобок и обработка вложенных выражений.

Дек позволяет добавлять и удалять с обоих концов. Узел двусвязного списка хранит значение, previous и next. Структура знает head и tail. Добавление слева меняет несколько ссылок независимо от длины списка. При удалении последнего узла нужно обнулить оба конца; это частый источник ошибок.

Вставка в начало массива требует сдвинуть все n ссылок и стоит $O(n)$. У списка она $O(1)$, но каждый узел требует дополнительной памяти, отдельного выделения и переходов по ссылкам. Поэтому оценка $O(1)$ не означает, что собственный Python-дек обязательно быстрее встроенной реализации. Встроенный collections.deque поддерживает быстрые операции на обоих концах.[2]

Операция	Динамический массив	Двусвязный дек
Доступ по индексу	$O(1)$	$O(n)$, в данном интерфейсе отсутствует
Добавление в конец	$O(1)$ амортизировано	$O(1)$
Добавление в начало	$O(n)$, в интерфейсе массива отсутствует	$O(1)$
Дополнительные затраты	Резерв свободных ячеек	Две ссылки на узел

Мастер-теорема без путаницы

Для $T(n) = aT(n/b) + f(n)$, $a \geq 1$, $b > 1$ сравнивают $f(n)$ с $n^{\log_b a}$. Если f полиномиально меньше, ответ $\Theta(n^{\log_b a})$. Если $f = \Theta(n^{\log_b a})$, ответ $\Theta(n^{\log_b a} \cdot \log n)$. Если f полиномиально больше и выполнено условие регулярности $a \cdot f(n/b) \leq c \cdot f(n)$, $c < 1$, ответ $\Theta(f(n))$.

Примеры: $2T(n/2) + 1 \rightarrow \Theta(n)$; $2T(n/2) + n \rightarrow \Theta(n \log n)$; $2T(n/2) + n^2 \rightarrow \Theta(n^2)$. Теорема не применяется напрямую к Фибоначчи или Ханоя: там размер уменьшается на 1, а не делится на постоянное b .

Запуск и проверка

```
python -m pytest labs/lab02 -q
python -m labs.lab02.experiment --variant 15
```

Тесты сверяют факториал с `math.factorial`, Фибоначчи — с последовательным эталоном, стек — с `list`, дек — с `collections.deque`. Отдельно проверяется переход ёмкости через 1024 элемента, пустые удаления и тысячи смешанных операций. Эксперимент измеряет среднюю стоимость добавления, количество рекурсивных вызовов и число ходов Ханоя.

Вопросы на защите и ответы

- 1. Зачем базовый случай?** Он завершает разложение задачи. Нужно также доказать, что аргумент приближается к нему, например n уменьшается на 1 и остаётся неотрицательным.
- 2. Какие три случая у мастер-теоремы?** Доминируют листья дерева рекурсии, все уровни одинаково или работа ближе к корню. Формальные условия и по одному примеру приведены выше; для третьего случая нельзя забывать регулярность.
- 3. Почему дорогое расширение не разрушает $O(1)$ append?** Расширения редкие, а сумма размеров копируемых буферов — геометрическая прогрессия. Оценка относится к последовательности, не к каждому отдельному вызову.
- 4. Почему связный список не всегда быстрее?** Одинаковая асимптотика не учитывает постоянные затраты, локальность памяти, выделение узлов и работу интерпретатора. Сравнивать нужно конкретные операции на одинаковых данных.
- 5. Чем стек отличается от дека?** Стек разрешает работу с одним концом; дек — с двумя. Стек можно реализовать через дек, но его ограниченный интерфейс лучше выражает LIFO.
- 6. Что именно ускоряет мемоизация?** Она убирает повторное вычисление одинаковых подзадач. Глубина рекурсии при этом остаётся $O(n)$, поэтому переполнение стека всё ещё возможно.
- 7. Почему в `hanoi` используется `yield`?** Он позволяет обрабатывать каждый ход сразу. Хранение всех $2^n - 1$ ходов потребовало бы экспоненциальной памяти.
- 8. Какие случаи особенно опасны для дека?** Удаление из пустой структуры, удаление единственного узла, чередование `appendleft/pop`. После каждой операции проверяем согласованность обратных ссылок и `size`.

Источники

[1] МТУСИ. КИМ-02, 2026.

[2] Python Software Foundation. collections — deque objects, официальная документация.

Отчёт по ЛР 2. Рекурсия и линейные структуры

Цель и выполненная работа

Реализованы алгоритмы и проверки, описанные в конспекте. Эксперимент выполнен на данных варианта 15, seed 45. В таблицах приведены фактические результаты запуска; время указано в секундах, если не оговорено иное.

Условия и методика

CPU: 12th Gen Intel(R) Core(TM) i7-12650H. ОС: Windows-11-10.0.26200-SP0. Python 3.12.12, NumPy 2.5.3, Matplotlib 3.11.1. Дата и время UTC: 2026-09-10T18:24:44.768238+00:00.

Один прогрев и пять повторов на точку, итог — медиана `time.perf_counter`. Ввод, генерация и подготовка свежего состояния исключены из таймера. Фоновые процессы и питание не контролировались; задержки рабочего компьютера могут влиять на отдельные точки. Контрольные суммы данных находятся в `environment.json`.

Результаты и их объяснение

Операция	n	с/операцию
<code>DynamicArray.append</code>	100000	3.27809e-07
<code>list.append</code>	100000	1.08334e-07
<code>Deque.appendleft</code>	100000	1.03213e-06
<code>deque.appendleft</code>	100000	1.00128e-07
<code>list.insert(0)</code>	100000	1.2653e-05

Сравнивается средняя стоимость одной операции в серии из `n` добавлений в изначально пустую структуру. `list.insert(0)` сдвигает растущий массив; остальные операции избегают полного сдвига. У собственного `DynamicArray` расширения видны в общей работе, но их стоимость распределяется по серии.

Фибоначчи	n	F(n)	Вызовы	Медиана, с
naive	5	5	15	2.70001e-06
memo	5	5	9	2.3e-06
naive	10	55	177	1.98e-05
memo	10	55	19	3.7e-06
naive	15	610	1973	0.0002341
memo	15	610	29	5.20001e-06
naive	20	6765	21891	0.002685
memo	20	6765	39	6.69999e-06
naive	25	75025	242785	0.0304263
memo	25	75025	49	8.39999e-06

Диски	Получено ходов	$2^n - 1$
1	1	1
2	3	3
3	7	7
4	15	15
5	31	31
6	63	63
7	127	127
8	255	255
9	511	511
10	1023	1023
11	2047	2047
12	4095	4095
13	8191	8191
14	16383	16383
15	32767	32767

При 1025 добавлениях ёмкость равна 2048, скопировано 2047 ссылок. Вместе с новыми записями это 3072 действия, меньше бюджета $3 \cdot 1025 = 3075$ в модели записи/копирования. Полная таблица переходов ёмкости сохранена в growth.csv. Мемоизация устраняет повторное вычисление одинаковых подзадач; все полученные числа ходов Ханоя совпали с формулой.

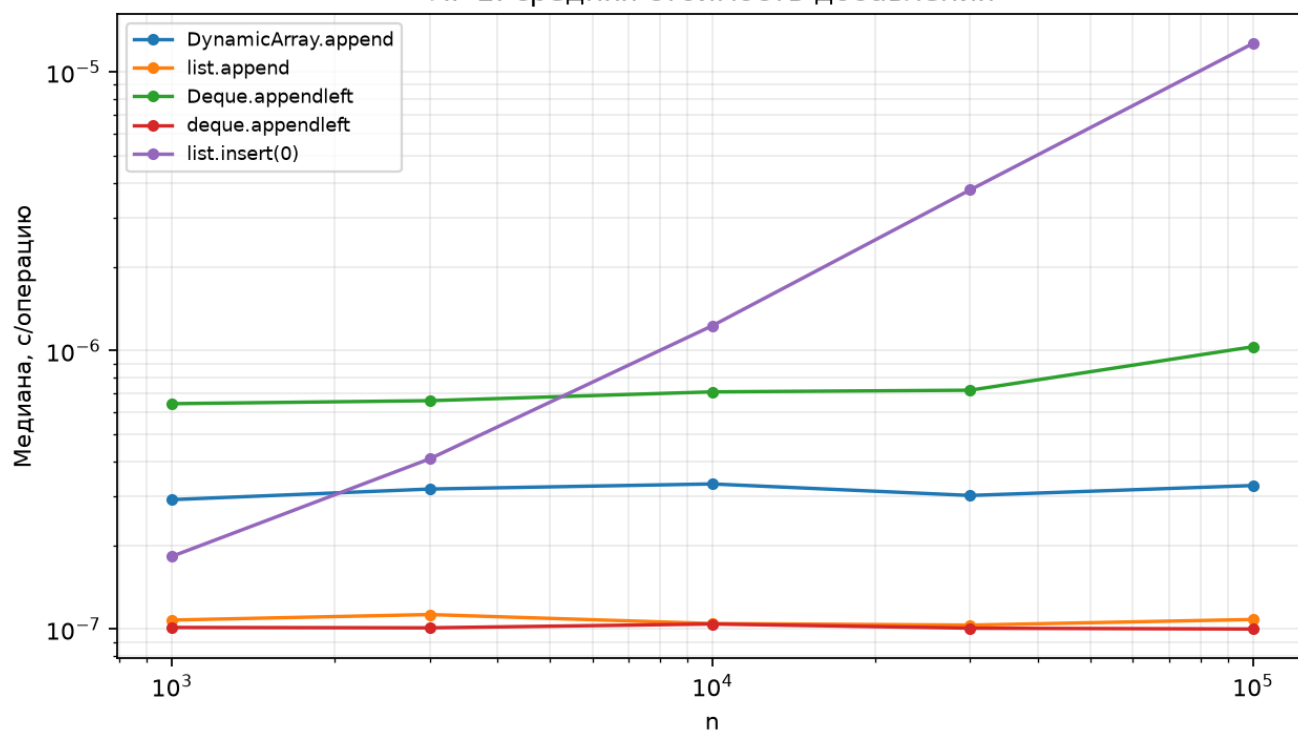
Проверка и вывод о применимости

Проверки находятся в test_lab.py. Запуск: `python -m pytest labs/lab02 -q`. Применены граничные случаи, инварианты и независимые эталоны; состав тестов подробно разобран в конспекте. Проверки всего комплекта также сохранены в verification.txt в корне репозитория.

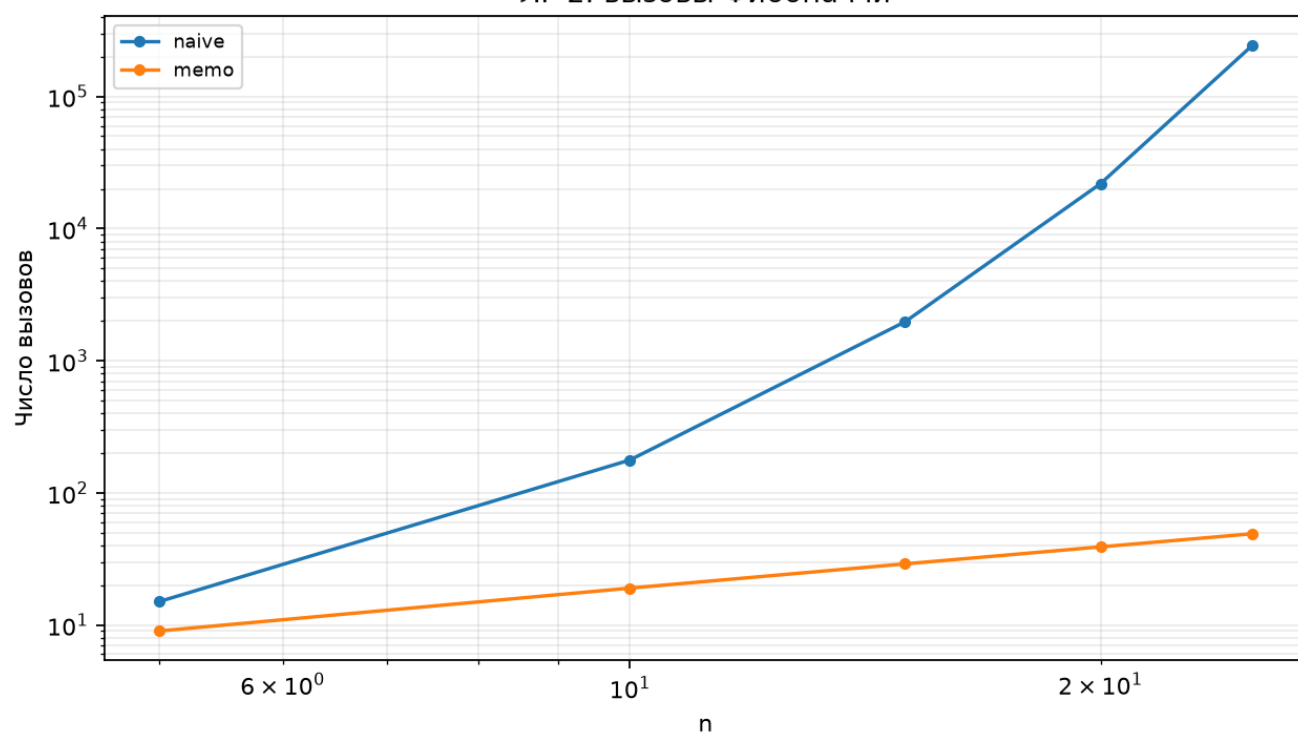
Вывод: принять для учебного использования в рамках явно описанных контрактов. Измерения подтверждают основные тенденции роста; ограничения реализаций и сравнения указаны выше и в конспекте. Автоматическая проверка не оценивает готовность к устной защите.

Графики

ЛР 2: средняя стоимость добавления



ЛР 2: вызовы Фибоначчи



Исходные данные отчёта

CSV содержит медиану и все пять длительностей trial_1...trial_5. Вспомогательные таблицы без времени содержат точные счётчики или свойства структуры.

- append.csv

- growth.csv
- hanoi.csv
- recursion.csv
- Паспорт окружения и контрольные суммы